

Graph Algorithms

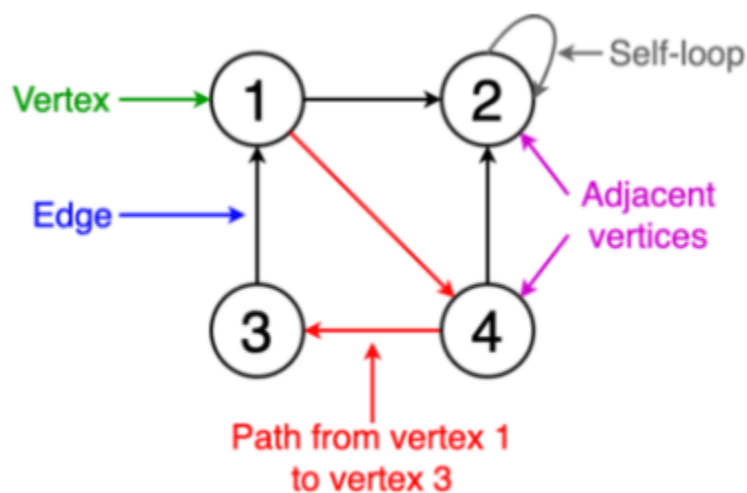
What is a Graph?

A graph is a **unique data structure** in programming that consists of finite sets of nodes or vertices and a set of edges that connect these vertices to them. At this moment, adjacent vertices can be called those vertices that are connected to the same edge with each other. In simple terms, a graph is a visual representation of vertices and edges sharing some connection or relationship. Although there are plenty of graph algorithms that you might have been familiar with, only some of them are put to use. The reason for this is simple as the standard graph algorithms are designed in such a way to solve millions of problems with just a few lines of logically coded technique. To some extent, one perfect algorithm is solely optimized to achieve such efficient results.

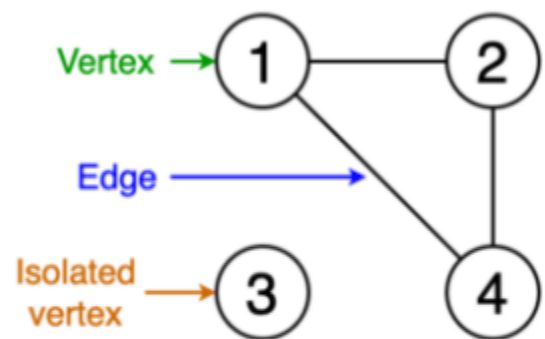
Types of Graphs

There are various types of graph algorithms that you would be looking at in this article but before that, let's look at some types of terms to imply the fundamental variations between them.

1. **Order:** Order defines the total number of vertices present in the graph.
2. **Size:** Size defines the number of edges present in the graph.
3. **Self-loop:** It is the edges that are connected from a vertex to itself.
4. **Isolated vertex:** It is the vertex that is not connected to any other vertices in the graph.
5. **Vertex degree:** It is defined as the number of edges incident to a vertex in a graph.
6. **Weighted graph:** A graph having value or weight of vertices.
7. **Unweighted graph:** A graph having no value or weight of vertices.
8. **Directed graph:** A graph having a direction indicator.
9. **Undirected graph:** A graph where no directions are defined.



Directed Graph



Undirected Graph

Max Flow Problem Introduction

The max flow problem is a classic optimization problem in graph theory that involves finding the maximum amount of flow that can be sent through a network of pipes, channels, or other pathways, subject to capacity constraints. The problem can be used to model a wide variety of real-world situations, such as transportation systems, communication networks, and resource allocation.

In the max flow problem, we have a directed graph with a source node s and a sink node t , and each edge has a capacity that represents the maximum amount of flow that can be sent through it. The goal is to find the maximum amount of flow that can be sent from s to t , while respecting the capacity constraints on the edges.

One common approach to solving the max flow problem is the Ford-Fulkerson algorithm, which is based on the idea of augmenting paths. The algorithm starts with an initial flow of zero, and iteratively finds a path from s to t that has available capacity, and then increases the flow along that path by the maximum amount possible. This process continues until no more augmenting paths can be found.

Another popular algorithm for solving the max flow problem is the Edmonds-Karp algorithm, which is a variant of the Ford-Fulkerson algorithm that uses breadth-first search to find augmenting paths, and thus can be more efficient in some cases.

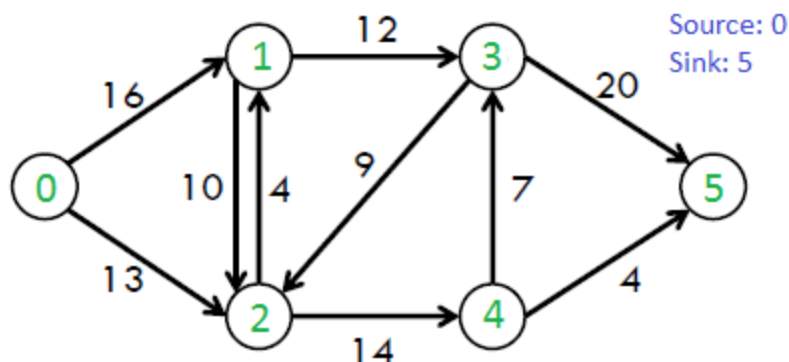
Advantages:

1. The max flow problem is a flexible and powerful modeling tool that can be used to represent a wide variety of real-world situations.
2. The Ford-Fulkerson and Edmonds-Karp algorithms are both guaranteed to find the maximum flow in a graph, and can be implemented efficiently for most practical cases.
3. The max flow problem has many interesting theoretical properties and connections to other areas of mathematics, such as linear programming and combinatorial optimization.

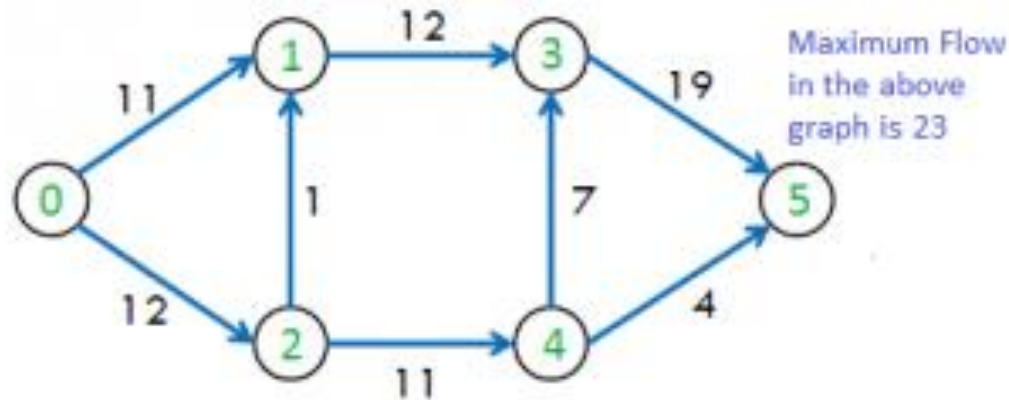
Disadvantages:

1. In some cases, the max flow problem can be difficult to solve efficiently, especially if the graph is very large or has complex capacity constraints.
2. The max flow problem may not always provide a unique or globally optimal solution, depending on the specific problem instance and algorithm used.

Maximum flow problems involve finding a feasible flow through a single-source, single-sink flow network that is maximum. Let's take an image to explain how the above definition wants to say.



Each edge is labeled with capacity, the maximum amount of stuff that it can carry. The goal is to figure out how much stuff can be pushed from the vertex s(source) to the vertex t(sink).



maximum flow possible is : 23

String Matching Introduction

String Matching Algorithm is also called "String Searching Algorithm." This is a vital class of string algorithm is declared as "this is the method to find a place where one or several strings are found within the larger string."

Given a text array, $T[1.....n]$, of n character and a pattern array, $P[1.....m]$, of m characters. The problems are to find an integer s , called **valid shift** where $0 \leq s < n-m$ and $T[s+1.....s+m] = P[1.....m]$. In other words, to find even if P in T , i.e., where P is a substring of T . The item of P and T are character drawn from some finite alphabet such as $\{0, 1\}$ or $\{A, BZ, a, b..... z\}$.

Given a string $T[1.....n]$, the **substrings** are represented as $T[i.....j]$ for some $0 \leq i \leq j \leq n-1$, the string formed by the characters in T from index i to index j , inclusive. This process that a string is a substring of itself (take $i = 0$ and $j = m$).

The **proper substring** of string $T[1.....n]$ is $T[1.....j]$ for some $0 < i \leq j \leq n-1$. That is, we must have either $i > 0$ or $j < m-1$.

Using these descriptions, we can say given any string $T[1.....n]$, the substrings are

1. $T[i.....j] = T[i] T[i+1] T[i+2].....T[j]$ for some $0 \leq i \leq j \leq n-1$.
And proper substrings are
1. $T[i.....j] = T[i] T[i+1] T[i+2].....T[j]$ for some $0 \leq i \leq j \leq n-1$.

Note: If $i > j$, then $T[i.....j]$ is equal to the empty string or null, which has length zero.

Algorithms used for String Matching:

There are different types of method is used to finding the string

1. The Naive String Matching Algorithm
2. The Rabin-Karp-Algorithm
3. Finite Automata
4. The Knuth-Morris-Pratt Algorithm
5. The Boyer-Moore Algorithm

The Naive String Matching Algorithm

The naive approach tests all the possible placement of Pattern P [1.....m] relative to text T [1.....n]. We try shift $s = 0, 1, \dots, n-m$, successively and for each shift s . Compare T [s+1.....s+m] to P [1.....m].

The naive algorithm finds all valid shifts using a loop that checks the condition $P [1.....m] = T [s+1.....s+m]$ for each of the $n - m + 1$ possible value of s .

NAIVE-STRING-MATCHER (T, P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. for $s \leftarrow 0$ to $n - m$
4. do if $P [1.....m] = T [s + 1.....s + m]$
5. then print "Pattern occurs with shift" s

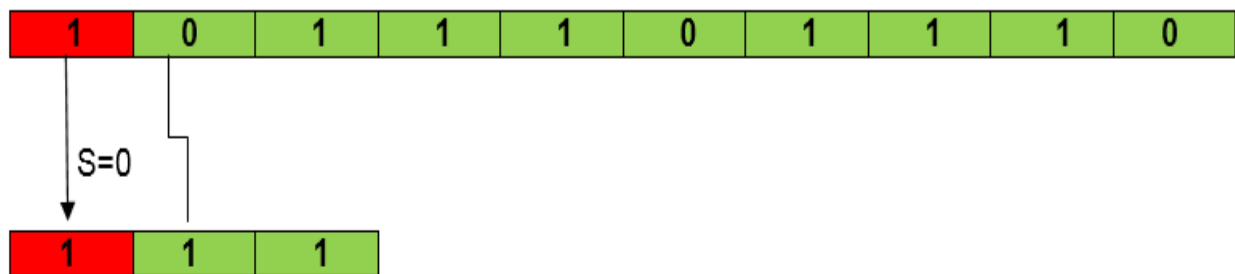
Analysis: This for loop from 3 to 5 executes for $n-m + 1$ (we need at least m characters at the end) times and in iteration we are doing m comparisons. So the total complexity is $O(n-m+1)$.

Example:

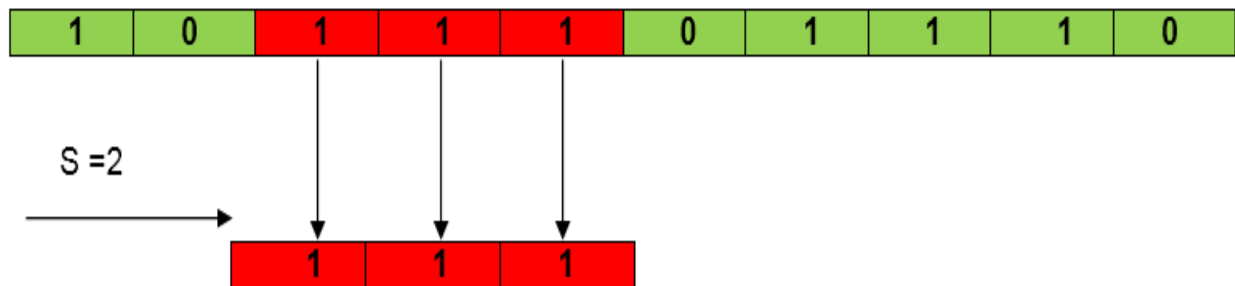
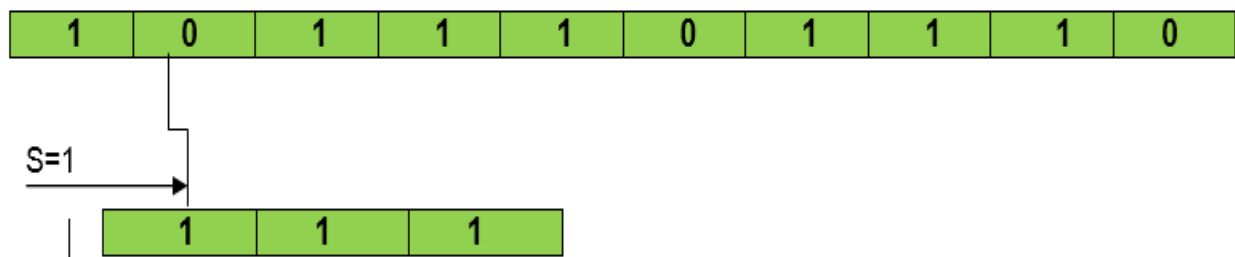
1. Suppose $T = 1011101110$
2. $P = 111$
3. Find all the Valid Shift

Solution:

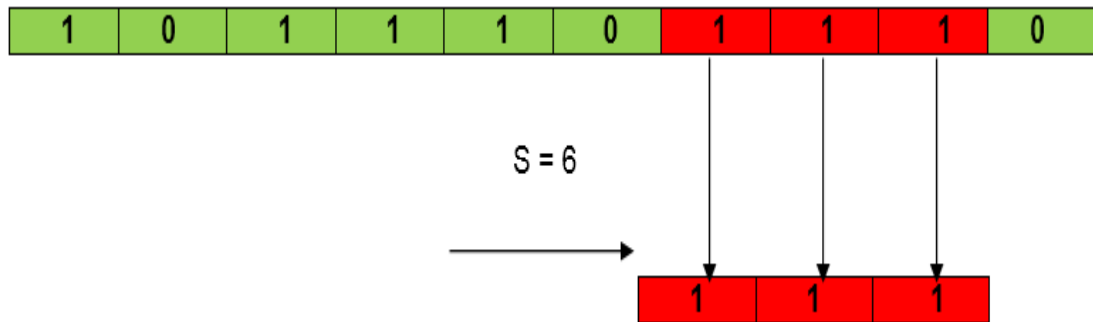
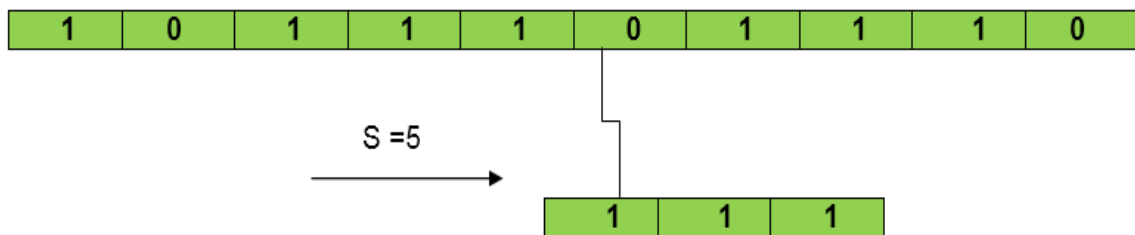
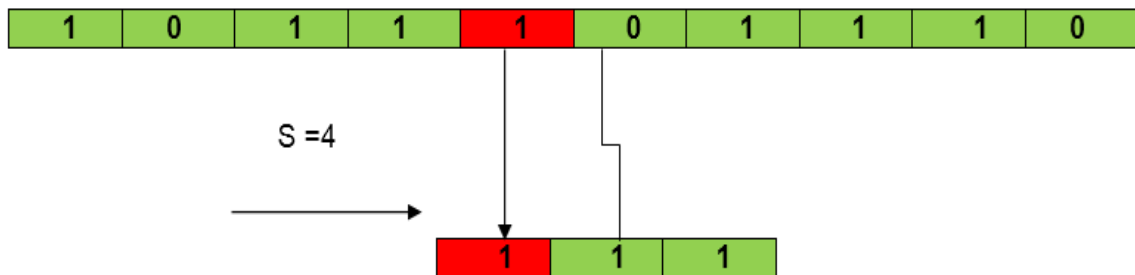
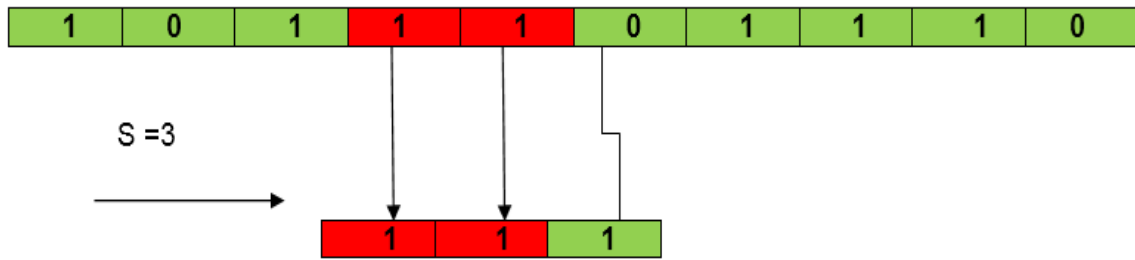
T = Text



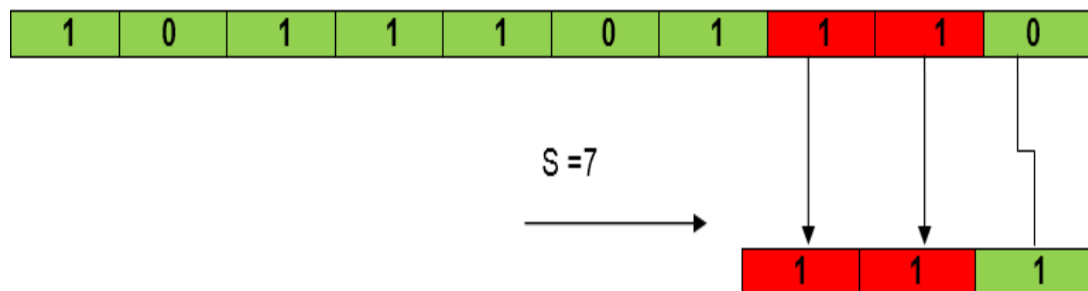
P = Pattern



So, $S=2$ is a Valid Shift



So, $S=6$ is a Valid Shift



String Matching with Finite Automata

String matching algorithms are techniques used in computer science and data analysis to find the occurrence or position of one string (the "pattern") within another string (the "text").

String Matching Algorithms

- Brute-Force Algorithm: This is the simplest method, where you compare the pattern with every substring of the text.
 - Knuth-Morris-Pratt (KMP) Algorithm: KMP is more efficient than brute force. It precompiles a longest prefix-suffix array for the pattern to avoid unnecessary comparisons, reducing the overall time complexity to $O(m + n)$.
 - Boyer-Moore Algorithm: Boyer-Moore uses a "bad character" and "good suffix" heuristic to skip sections of the text, making it very efficient in practice, especially for long texts and patterns.
 - Rabin-Karp Algorithm: Rabin-Karp uses hashing to quickly compare the pattern with potential substrings of the text. It has a time complexity of $O(m * n)$ in the worst case but can be very efficient for certain inputs.
 - Aho-Corasick Algorithm: This algorithm is used for searching multiple patterns simultaneously in a text. It constructs a finite automaton and efficiently identifies all occurrences of the patterns.
 - Regular Expressions: Many programming languages provide regular expression libraries for string matching. Regular expressions allow for complex pattern matching based on a specified pattern language.
-

Types of Finite Automata

1. Deterministic Finite Automaton (DFA): In a DFA, for each state and input symbol, there is exactly one transition to another state. It means that the behavior of the automaton is fully determined by its current state and the input symbol. DFAs are often used for tasks like recognizing regular languages, such as simple pattern matching in text processing.
2. Nondeterministic Finite Automaton (NFA): In an NFA, there can be multiple possible transitions from a state on a given input symbol. This non-determinism allows for greater flexibility but requires more complex algorithms to process. NFAs are often used as theoretical constructs and can be converted into equivalent DFAs using algorithms like the power set construction.

Key concepts and components of finite automata include:

- States: Finite automata have a finite set of states that represent different configurations or situations of the system being modeled.
 - Transitions: Transitions define how the automaton moves from one state to another in response to input symbols.
 - Start State: It's the initial state from which the automaton begins its operation.
 - Accepting (or Final) States: These states indicate successful or valid outcomes. In some applications, reaching an accepting state signifies that a specific pattern or input was recognized.
-

Finite automata have various applications in computer science, including:

- Lexical analysis in compiler design: Recognizing keywords, identifiers, and symbols in programming languages.
- Pattern matching in text processing and searching.
- Model checking in hardware and software verification.
- Natural language processing and parsing.

String matching using finite automata is a technique to find occurrences of a pattern (substring) within a text (string). It involves constructing a finite automaton (usually a Deterministic Finite Automaton or DFA) based on the pattern and then using this automaton to search for the pattern in the text efficiently.

Here are the basic steps:

1. Construct DFA: Build a DFA that represents the pattern you want to search for. This DFA will have states and transitions based on the characters in the pattern.
2. Search with DFA: Start at the initial state of the DFA and process each character of the text one by one. Follow the transitions in the DFA based on the characters encountered.
3. Match Found: If you reach an accepting state in the DFA while processing the text, it means you've found a match of the pattern in the text.

This technique can significantly speed up string matching compared to naive methods like brute force searching, especially when searching for the same pattern in multiple texts.

Keep in mind that constructing the DFA can have a time complexity of $O(m*|\Sigma|)$ where 'm' is the length of the pattern and $|\Sigma|$ is the size of the alphabet, but once the DFA is built, searching for the pattern in a text is very efficient, usually with a time complexity of $O(n)$, where 'n' is the length of the text.

Practical Applications of String-Matching Algorithm

1. Text Search: In search engines like Google, when you enter a query, string matching algorithms are used to quickly scan through billions of web pages and identify those that contain the exact or approximate string of characters you've entered. These algorithms often implement optimizations like indexing to speed up the search process by pre-sorting and organizing the data.
2. Data Retrieval: In a database management system, string matching can be used in SQL queries with the LIKE operator to retrieve records that match a specified pattern. For example, you can use `SELECT * FROM Customers WHERE LastName LIKE 'Sm%'` to retrieve customers with last names starting with 'Sm.' String indexing and hashing techniques are employed to improve the efficiency of such operations.

3. **Spell Checking and Correction:** Spell checkers use string matching to compare input words to a dictionary of correctly spelled words. The algorithm suggests corrections based on the closest matches found in the dictionary. Levenshtein distance, a string distance metric, is often used to measure the similarity between two strings and recommend corrections accordingly.
4. **Pattern Recognition:** In DNA sequence analysis, string matching is crucial for identifying specific patterns, such as genes or regulatory elements, within a DNA sequence. In image processing, string matching can be used to detect patterns or templates within an image, aiding in tasks like object recognition.
5. **Regular Expressions:** Regular expressions are a powerful tool for pattern matching and manipulation of strings in text. They are used in programming languages and text editors for tasks like searching, extracting, and replacing text that matches a specified pattern.
6. **Network Security:** Intrusion detection systems use string matching to compare network traffic against known attack patterns or signatures. If a match is found, it can trigger alerts or actions to mitigate the threat. Signature-based antivirus software also employs string matching to identify malware signatures in files.
7. **Bioinformatics:** In bioinformatics, string matching is used for sequence alignment. Algorithms like the Smith-Waterman and Needleman-Wunsch algorithms compare biological sequences (DNA, RNA, proteins) to find similarities, which can reveal evolutionary relationships and functional motifs.
8. **Web Scraping:** Web scraping tools use string matching to locate specific elements within the HTML source code of web pages.
9. **Geographic Information Systems (GIS):** GIS applications use string matching to search for geographical data, such as locations, landmarks, or addresses. This helps users locate and analyze spatial information on maps.
10. **Music Recognition:** In music analysis, string matching can be used to compare a short musical pattern (like a melody or chord progression) to a database of known musical patterns. This is often used in music recognition apps to identify songs or provide chord suggestions.

Complexity of String Matching using Finite Automata

String matching using finite automata is an efficient and widely used technique with a relatively low computational complexity, especially when you preprocess the pattern to create a finite automaton. The key advantage is that the pattern preprocessing is done once, making subsequent searches in text strings very fast.

1. Preprocessing (Building the Finite Automaton):

Time Complexity: $O(m^3 * |\Sigma|)$, where m is the length of the pattern and Σ is the alphabet (set of possible characters).

Space Complexity: $O(m * |\Sigma|)$, where the finite automaton requires space proportional to the size of the pattern and the alphabet.

In practice, the actual time and space complexity may be lower due to optimizations like the Aho-Corasick algorithm or efficient representations of finite automata.

2. Searching (Using the Finite Automaton):

Time Complexity: $O(n)$, where n is the length of the text string.

Space Complexity: $O(1)$, as the memory requirements do not depend on the size of the input text.

The key insight is that the time complexity for searching (using the finite automaton) is linear with respect to the length of the text string. This is a significant advantage for string matching when multiple searches are performed on different text strings using the same pattern because the preprocessing step is done only once.

Finite Automata over other Algorithms for String Matching

1. **Determinism and Simplicity:** DFAs are simple and deterministic state machines, making them easy to understand and implement. This simplicity leads to shorter code and lower chances of implementation errors.
2. **Efficient Preprocessing:** DFAs have a preprocessing step (building the automaton) that operates in $O(m^3 * |\Sigma|)$ time, where m is the length of the pattern and Σ is the alphabet. While this can seem slower than some other algorithms in the preprocessing phase, it's a one-time cost. Once the automaton is built, searching for the pattern in multiple text strings is extremely efficient, with a time complexity of $O(n)$ per search, where n is the length of the text. This is especially advantageous in scenarios where you need to search for the same pattern in multiple texts.
3. **Memory Efficiency:** Finite automata have a memory complexity of $O(m * |\Sigma|)$ for storing the automaton, which is relatively low and doesn't depend on the size of the input text. In contrast, some other algorithms like the Boyer-Moore algorithm require additional memory to construct auxiliary data structures for pattern preprocessing.
4. **Deterministic Search Time:** The search time using finite automata is deterministic and linear with respect to the length of the text. In the worst case, the entire text is examined, but there are no backtrackings or worst-case scenarios where the search time becomes quadratic.
5. **Parallelism:** Finite automata can be parallelized efficiently because each character in the text can be processed independently. This makes them suitable for parallel processing architectures and multi-core CPUs.
6. **Handling Multiple Patterns:** Finite automata can efficiently search for multiple patterns simultaneously by constructing a single automaton that recognizes all the patterns.
7. **Fixed-Pattern Matching:** Finite automata are particularly useful when you are searching for fixed patterns (patterns that do not change frequently), as the preprocessing cost can be amortized over multiple searches.

Limitations of Finite Automata for String Matching

1. **Limited Memory:** Finite automata have a finite number of states, and they transition from one state to another based on the input characters. Once a character is processed and the automaton transitions to a new state, it typically forgets the previous states. This lack of memory means that finite automata cannot remember information about past characters or their positions in the input string.
2. **Inability to Handle Complex Patterns:** Finite automata are well-suited for recognizing simple patterns or regular languages. They excel at tasks like recognizing whether a string follows a specific regular expression pattern (e.g., recognizing valid email addresses or phone numbers). However, for more complex patterns or languages that require remembering previous characters or matching nested constructs (e.g., balanced parentheses), finite automata are insufficient.
3. **Limited Expressiveness:** Finite automata can only recognize regular languages, which are a subset of the formal language hierarchy. Regular languages are limited in expressive power compared to context-free, context-sensitive, and recursively enumerable languages. This means that finite automata cannot handle certain advanced string-matching tasks that require context-sensitive or context-free rules.
4. **Performance Limitations:** For string matching tasks where patterns have many possible variations or long patterns, the number of states in a DFA or transitions in an NFA can grow significantly. This can result in large automata with many states and transitions, which may lead to increased memory and processing time requirements.
5. **Exact Matches Only:** Finite automata are primarily used for exact string matching, where the goal is to determine whether a given string matches a specified pattern exactly. They are not well-suited for tasks that require approximate or fuzzy matching, where slight variations or errors in the input string need to be tolerated.
6. **Complexity of Building Automata:** Constructing DFAs or NFAs for certain string-matching tasks can be challenging and may require a deep understanding of regular expressions and automata theory. Creating automata for complex patterns can result in intricate and hard-to-maintain state diagrams.

Alternatives of Finite Automata for String Matching

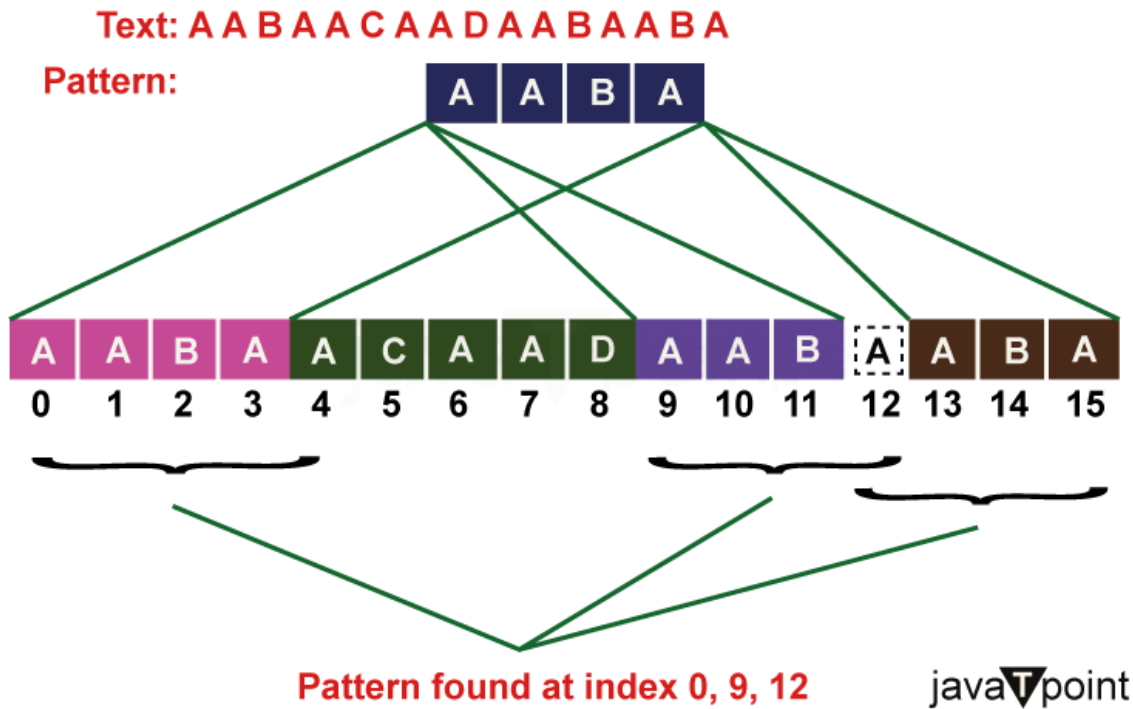
1. **Regular Expressions:** Regular expressions (regex) are a powerful tool for pattern matching. They use a syntax to specify patterns that can match strings. Regex engines can vary in efficiency. Simple patterns can be matched quickly, but complex patterns or those with excessive backtracking may lead to performance issues, especially for large inputs.
2. **Knuth-Morris-Pratt (KMP) Algorithm:** The KMP algorithm is an efficient algorithm for exact string matching. It uses a preprocessed pattern to skip unnecessary comparisons during matching. KMP has a time complexity of $O(m + n)$, where m is the length of the pattern and n is the length of the input string. It performs well, especially for long patterns.
3. **Boyer-Moore Algorithm:** The Boyer-Moore algorithm is another efficient algorithm for exact string matching. It uses a heuristic to skip ahead in the input string during matching. Boyer-Moore has an average-case time complexity of $O(n/m)$ for searching an

n-character string with an m-character pattern. It performs very well in practice, especially for longer patterns.

4. Rabin-Karp Algorithm: The Rabin-Karp algorithm uses hashing to efficiently search for a pattern in a text. It can also be adapted for approximate matching. Rabin-Karp has an average-case time complexity of $O((n-m+1)m)$ for searching an n-character string with an m-character pattern. Its performance can be competitive with other algorithms for certain patterns and inputs.
5. Aho-Corasick Algorithm: The Aho-Corasick algorithm is designed for multiple string matching, allowing the search for multiple patterns simultaneously. Aho-Corasick has a time complexity of $O(n + z + m)$, where n is the length of the input string, m is the total length of patterns, and z is the number of matches found. It is efficient for multiple pattern matching tasks.
6. Trie Data Structure: A trie is a tree-like data structure used for dictionary-based string matching, often in spell checking or autocomplete. Tries are efficient for dictionary-based matching, with a time complexity of $O(m)$ for searching for a string of length m. They can be space-efficient when there are many common prefixes in the patterns.
7. Suffix Trees and Suffix Arrays: Suffix trees and suffix arrays are versatile data structures for various string processing tasks, including string matching. They can support efficient substring search, approximate matching, and other advanced string algorithms. Their construction can take $O(n)$ time and space, but they enable efficient queries afterward for a wide range of string processing tasks.

The Knuth Morris Pratt (KMP) Algorithm

Finding and locating specific patterns or substrings within a text or a larger string is one of the fundamental tools used in computer science. These functionalities are useful as to find a particular keyword or text within a larger text as doing the same manually would be a tedious task.



What is String-searching Algorithms?

String-searching algorithms are a way to efficiently look for a given substring or a specific pattern within a longer string. These patterns could be a single character, simple words, complex words or even phrases.

Importance in Computer Science

- Text Processing: String-searching algorithms are at the core of text processing tasks, such as searching for keywords in documents, parsing data, and extracting relevant information from textual data sources.
- Data Retrieval: In databases, string-searching is essential for querying information efficiently. It enables the retrieval of records that contain specific data patterns, improving the speed of data retrieval.
- Compiler Design: Compilers use string-searching algorithms to recognize and parse programming language constructs within source code, aiding in the compilation process.
- Bioinformatics: In bioinformatics, these algorithms help identify genetic sequences, patterns in DNA, and protein motifs, facilitating important research in genomics and proteomics.
- Network Security: String-searching is crucial in intrusion detection systems, where patterns of malicious code or network behaviors need to be identified rapidly.
- Natural Language Processing (NLP): NLP applications rely on string-searching to perform tasks like sentiment analysis, named entity recognition, and machine translation.
- Web Search Engines: Search engines use advanced string-searching techniques to match user queries to web pages, providing relevant search results.

Significance

- Efficiency: String-searching algorithms are designed for efficiency. They aim to minimize the time and resources required to locate patterns within large datasets. Efficient algorithms are critical for applications with strict performance requirements.
 - Algorithmic Diversity: There is a wide variety of string-searching algorithms, each designed for specific scenarios and data characteristics.
 - Problem-Solving: String-searching algorithms exemplify algorithmic problem-solving, an essential skill in computer science. They provide insight into techniques for optimizing data retrieval and pattern-matching tasks.
 - Interdisciplinary Impact: String-searching algorithms have applications in numerous interdisciplinary fields, fostering collaboration between computer scientists and researchers in biology, linguistics, and more.
 - Continual Advancements: The field of string-searching algorithms continues to evolve, with researchers developing new algorithms and optimizations to address emerging challenges in data processing.
-

Significance of Efficient String-Searching Algorithm

The significance of efficient string-searching algorithms lies in their ability to address critical computational challenges and provide practical solutions in various fields. Here are some key points highlighting the importance of efficient string-searching algorithms:

- Time Complexity Reduction: Efficient algorithms drastically reduce the time required to search for patterns within large text or data sets.
- Resource Optimization: Efficient string-searching algorithms minimize the use of computational resources, including CPU time and memory. This resource optimization is critical for applications running on resource-constrained devices, such as mobile phones and embedded systems.
- Improved User Experience: In user-facing applications like web search engines and data retrieval systems, faster response times lead to improved user experiences. Users are more likely to engage with applications that provide quick and relevant results.
- Scalability: Efficient algorithms are scalable, meaning they can handle increasingly large datasets without a proportional increase in execution time. This scalability is vital in handling the ever-growing volumes of data generated in today's digital world.
- Enhanced Productivity: In software development and data analysis, efficient string-searching algorithms enable developers and analysts to build and deploy applications faster. They reduce the time and effort required for optimizing search operations.
- Reduced Energy Consumption: Energy-efficient algorithms are critical for battery-powered devices. By minimizing computational work, efficient string-searching algorithms help extend the battery life of smartphones, tablets, and IoT devices.
- Cost Savings: In cloud computing environments, where computational resources are billed based on usage, efficient algorithms can lead to significant cost savings by reducing the amount of computation required for search and data retrieval tasks.
- Security and Intrusion Detection: In the realm of cyber security, efficient string-searching algorithms are essential for quickly identifying and responding to threats. They enable

intrusion detection systems to recognize patterns associated with malware and malicious network activity.

- Scientific Discovery: In fields like genomics and bioinformatics, efficient pattern-matching algorithms help researchers identify genetic sequences, motifs, and patterns. This accelerates the pace of scientific discovery in areas like drug design and disease understanding.
 - Competitive Advantage: Companies and organizations that can efficiently process and search through vast amounts of data gain a competitive advantage.
 - Algorithmic Research: Efficient string-searching algorithms are a subject of ongoing algorithmic research. The pursuit of better algorithms continues to drive innovation and the development of new techniques, benefiting computer science as a whole.
-

History and Background of KMP Algorithm

The Knuth-Morris-Pratt (KMP) algorithm is a string-searching algorithm that was developed by Donald Knuth, Vaughan Pratt, and James H. Morris. It was introduced in 1977 and has since become a fundamental algorithm in computer science and string processing.

Background:

Before the KMP algorithm was developed, the "brute-force" or "naive" method was commonly used to execute string searches. In the brute-force approach, a pattern would be slid over a text character by character, with each point being checked for matches. This method was ineffective for long texts and patterns since it had a time complexity of $O(m * n)$, where m is the length of the pattern and n is the length of the text.

Development and History:

- Donald Knuth's Influence: Donald Knuth, a prominent computer scientist and the author of "The Art of Computer Programming," contributed to the development of the KMP algorithm. He recognized the limitations of the brute-force technique for pattern matching and hence aimed to find a more time and space efficient solution with a better accuracy.
 - Vaughan Pratt and James H. Morris: Vaughan Pratt and James H. Morris, along with Knuth, collaborated on the research and development of the KMP algorithm. Each of them made significant contributions to the algorithm's design and analysis.
 - Publication: The KMP algorithm was officially introduced in the paper titled "Fast Pattern-matching in Strings" published in 1977 by Knuth and Pratt. This paper outlined the algorithm's core principles and its efficient approach to string-searching.
-

Key Contributions:

The KMP algorithm's primary innovation is the use of a "failure function" or "partial match table," which is recomputed from the pattern. This table allows the algorithm to skip unnecessary comparisons when a mismatch occurs during the search, making it more efficient than brute-force methods.

Why is KMP Preferred over other String-searching Algorithms?

- Efficiency: KMP is designed to operate in linear time complexity, with a worst-case time complexity of $O(m + n)$, where 'm' is the length of the pattern and 'n' is the length of the text. The time complexity of the KMP algorithm makes it faster than the brute-force techniques which have a worst-case complexity of $O(m*n)$.
 - Linear Complexity: KMP's linear time complexity remains consistent regardless of the size of the input text or pattern. This property makes it suitable for processing large datasets efficiently, making it a preferred choice in applications dealing with extensive textual data.
 - Avoids Redundant Comparisons: KMP utilizes a "failure function" or "partial match table" that allows it to skip unnecessary character comparisons when a mismatch occurs. This mechanism minimizes redundant comparisons and contributes to its speed.
 - Robustness: KMP handles patterns with repeated characters gracefully. It can efficiently handle patterns with repeated substrings, which can pose challenges for some other algorithms.
 - Memory Efficiency: KMP's memory usage is relatively low, as it primarily relies on the construction of the partial match table.
 - Applicability: KMP can be applied to a wide range of string-searching tasks, from simple keyword searches in documents to more complex pattern-matching in genomic sequences, source code analysis, and network intrusion detection.
 - Algorithmic Diversity: While KMP is highly efficient, there are variations and optimizations of the algorithm that can be tailored to specific use cases. This adaptability allows developers to choose the most suitable version for their particular requirements.
 - Worst-Case Guarantees: KMP provides worst-case time complexity guarantees, ensuring predictable performance in any situation. This predictability is essential in mission-critical applications where performance is non-negotiable.
 - Educational Value: KMP serves as a valuable educational tool for teaching algorithmic concepts like string-searching, pattern-matching, and the use of partial match tables. Its clear and efficient design makes it a useful teaching example.
 - Historical Significance: KMP is a classic algorithm with a rich history in computer science. Its development and analysis have contributed to the advancement of algorithmic research.
-

The Boyer-Moore Algorithm

Robert Boyer and J Strother Moore established it in 1977. The B-M String search algorithm is a particularly efficient algorithm and has served as a standard benchmark for string search algorithm ever since.

The B-M algorithm takes a 'backward' approach: the pattern string (P) is aligned with the start of the text string (T), and then compares the characters of a pattern from right to left, beginning with rightmost character.

If a character is compared that is not within the pattern, no match can be found by analyzing any further aspects at this position so the pattern can be changed entirely past the mismatching character.

For deciding the possible shifts, B-M algorithm uses two preprocessing strategies simultaneously. Whenever a mismatch occurs, the algorithm calculates a variation using both approaches and selects the more significant shift thus, if make use of the most effective strategy for each case.

The two strategies are called heuristics of B - M as they are used to reduce the search. They are:

1. Bad Character Heuristics
2. Good Suffix Heuristics

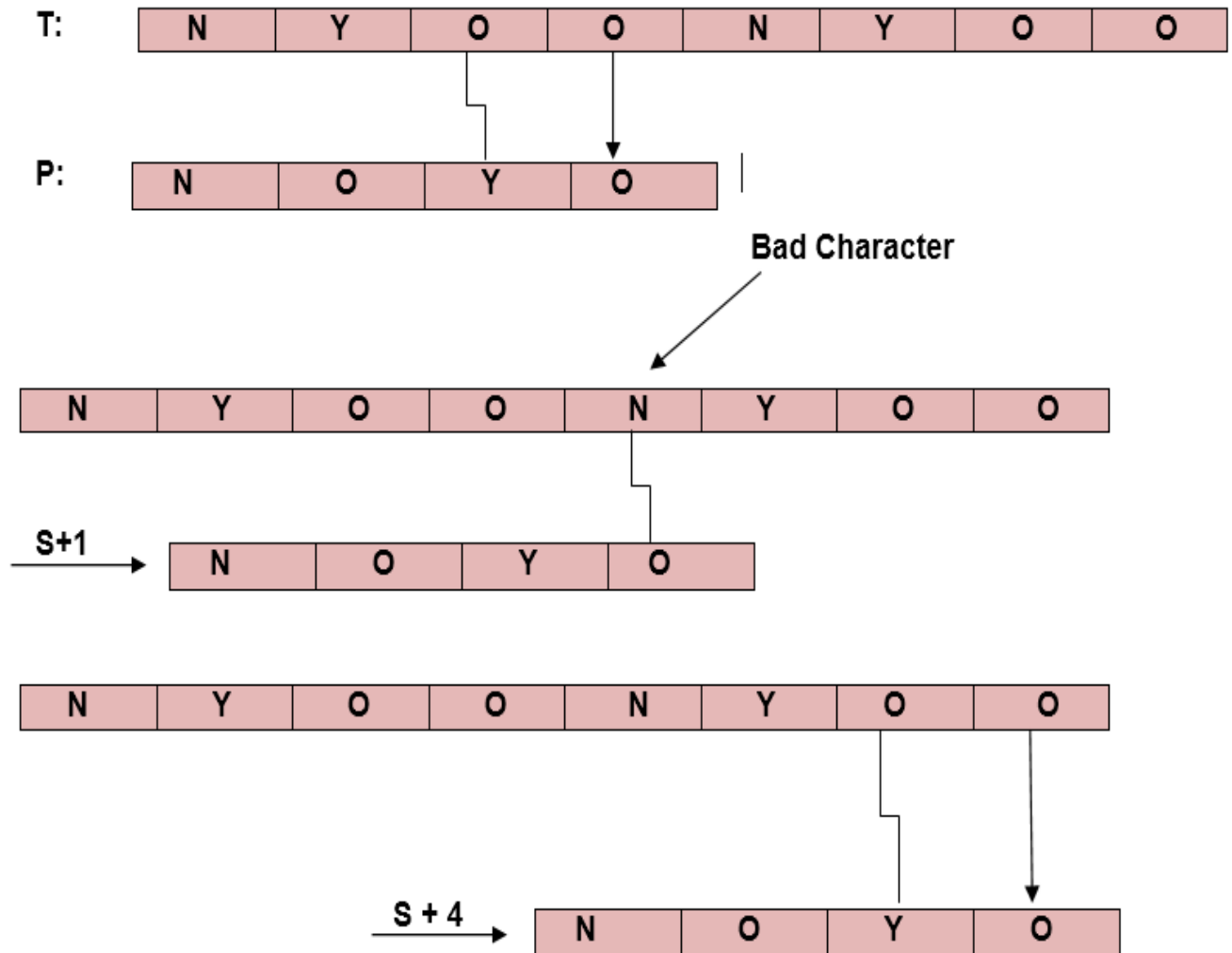
1. Bad Character Heuristics

This Heuristics has two implications:

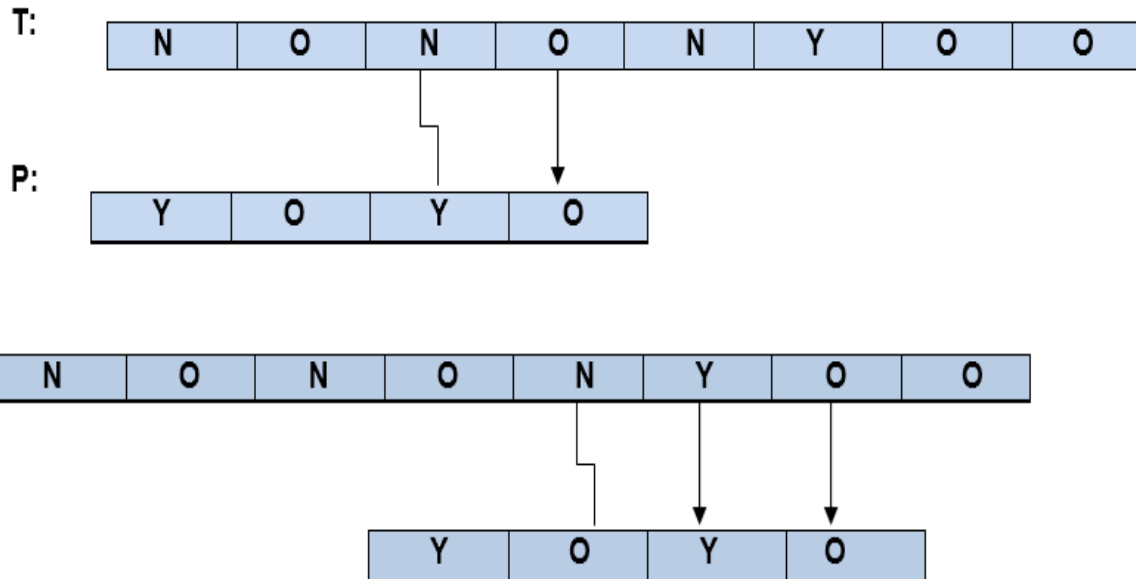
- Suppose there is a character in a text in which does not occur in a pattern at all. When a mismatch happens at this character (called as bad character), the whole pattern can be changed, begin matching from substring next to this 'bad character.'
- On the other hand, it might be that a bad character is present in the pattern, in this case, align the nature of the pattern with a bad character in the text.

Thus in any case shift may be higher than one.

Example1: Let Text T = <nyoo nyoo> and pattern P = <noyo>



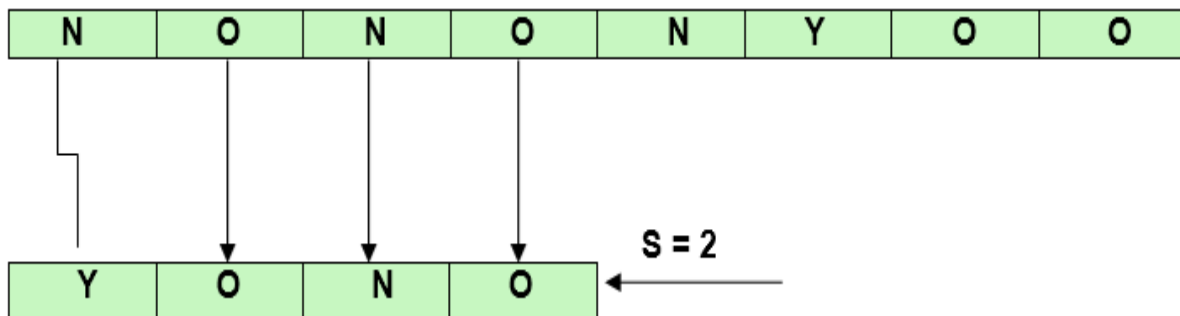
Example2: If a bad character doesn't exist the pattern then.



Problem in Bad-Character Heuristics:

In some cases, Bad-Character Heuristics produces some negative shifts.

For Example:



This means that we need some extra information to produce a shift on encountering a bad character. This information is about the last position of every aspect in the pattern and also the set of characters used in a pattern (often called the alphabet Σ of a pattern).

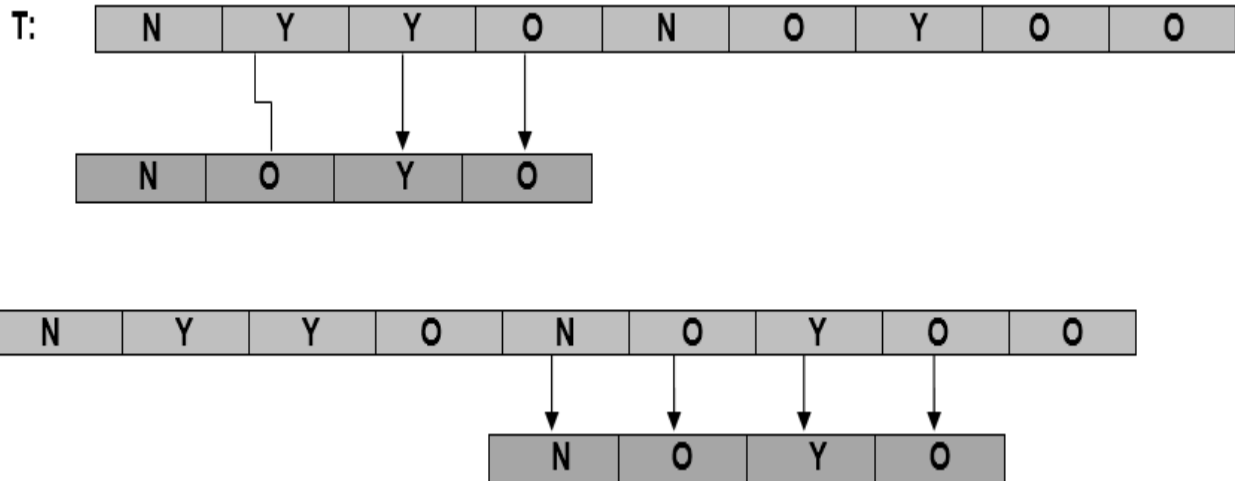
COMPUTE-LAST-OCCURRENCE-FUNCTION (P, m, Σ)

1. for each character $a \in \Sigma$
2. do $\lambda[a] = 0$
3. for $j \leftarrow 1$ to m
4. do $\lambda[P[j]] \leftarrow j$
5. Return λ

2. Good Suffix Heuristics:

A good suffix is a suffix that has matched successfully. After a mismatch which has a negative shift in bad character heuristics, look if a substring of pattern matched till bad character has a good suffix in it, if it is so then we have an onward jump equal to the length of suffix found.

Example:



COMPUTE-GOOD-SUFFIX-FUNCTION (P, m)

1. $\Pi \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P)$
2. $P' \leftarrow \text{reverse}(P)$
3. $\Pi' \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P')$
4. for $j \leftarrow 0$ to m
5. do $\gamma[j] \leftarrow m - \Pi[m]$
6. for $l \leftarrow 1$ to m
7. do $j \leftarrow m - \Pi'[L]$
8. If $\gamma[j] > l - \Pi'[L]$
9. then $\gamma[j] \leftarrow l - \Pi'[L]$
10. Return γ

BOYER-MOORE-MATCHER (T, P, Σ)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\lambda \leftarrow \text{COMPUTE-LAST-OCCURRENCE-FUNCTION}(P, m, \Sigma)$
4. $\gamma \leftarrow \text{COMPUTE-GOOD-SUFFIX-FUNCTION}(P, m)$
5. $s \leftarrow 0$
6. While $s \leq n - m$
7. do $j \leftarrow m$
8. While $j > 0$ and $P[j] = T[s + j]$

```

9. do  $j \leftarrow j-1$ 
10. If  $j = 0$ 
11. then print "Pattern occurs at shift"  $s$ 
12.  $s \leftarrow s + y[0]$ 
13. else  $s \leftarrow s + \max(y[j], j - \lambda[T[s+j]])$ 

```

Complexity Comparison of String Matching Algorithm:

Algorithm	Preprocessing Time	Matching Time
Naive	O	$(O(n - m + 1)m)$
Rabin-Karp	$O(m)$	$(O(n - m + 1)m)$
Finite Automata	$O(m \Sigma)$	$O(n)$
Knuth-Morris-Pratt	$O(m)$	$O(n)$
Boyer-Moore	$O(\Sigma)$	$(O((n - m + 1) + \Sigma))$